Introduction to the ER Model

The **Entity-Relationship (ER) model** is a high-level conceptual data model used to design database schemas.[1] It provides a way to represent the real world as a collection of interconnected objects and their relationships.[2] ER diagrams are a graphical representation of the ER model, making it a powerful tool for database design and communication between designers and users.[3]

Core Components of the ER Model

The ER model is built upon three fundamental concepts: **entities**, **attributes**, and **relationships**.[4]

1. Entities

An **entity** is a real-world object or concept that is distinguishable from other objects.[5] It represents a "thing" about which data needs to be stored.[6] An entity can be a person, place, object, event, or concept. Entities are represented in an ER diagram by **rectangles**.[7]

- **Entity Set**: A collection of similar entities.[8] For example, "Student" is an entity set, which would include individual entities like "John," "Sarah," and "Alex."[9]

There are two types of entities:

- **Strong Entity**: An entity that has a primary key and can be uniquely identified on its own.[10] It is represented by a single rectangle.
- **Weak Entity**: An entity that does not have a primary key of its own and depends on a strong entity for its unique identification.[11] It is represented by a **double rectangle**.[12]

2. Attributes

An **attribute** is a property or characteristic of an entity.[13] It describes the data stored within an entity. Attributes are represented by **ovals** and are connected to the entity rectangle by a line.[14]

- **Simple Attribute**: An attribute that cannot be broken down into smaller components (e.g., Age).[15]
- **Composite Attribute**: An attribute that can be divided into smaller sub-attributes (e.g., `Address` can be broken down into `Street`, `City`, and `ZipCode`).[16]
- **Derived Attribute**: An attribute whose value can be calculated from other attributes (e.g., `Age` can be derived from `DateOfBirth`).[17] It is represented by a **dashed oval**.
- **Key Attribute**: An attribute (or set of attributes) that uniquely identifies an entity within an entity set.[18] The key attribute is **underlined** in an ER diagram.[19]

3. Relationships

A **relationship** describes how two or more entities are associated with each other.[20] Relationships are represented by **diamonds**.

- **Relationship Set**: A collection of similar relationships.[21] For example, the relationship "enrolled in" connects the entity sets "Student" and "Course."[22]

ER Diagram: A Simple Example

Let's consider a simple database for a university.

- **Entities**:
  - **Student** (Strong Entity)[23]
  - **Course** (Strong Entity)
- **Attributes**:
  - **Student**: `StudentID` (key attribute), `Name`, `Age`, `Address` (composite attribute).
  - **Course**: `CourseID` (key attribute), `Title`, `Credits`.
- **Relationship**:
  - **Enrolled_In**: A relationship that links the `Student` entity to the `Course` entity.[24]

Cardinality Constraints

Cardinality defines the number of instances of one entity that can be associated with the number of instances of another entity via a relationship.[25] Cardinality constraints are represented by numbers on the lines connecting entities to the relationship diamond.[26]

The three main types of cardinalities are:

1. **One-to-One (1:1)**: An instance of entity A is related to at most one instance of entity B, and vice versa.[27]
   - Example: A `Student` may have at most one `Student_ID_Card`, and a `Student_ID_Card` belongs to at most one `Student`.
2. **One-to-Many (1:N)**: An instance of entity A can be related to many instances of entity B, but an instance of entity B is related to at most one instance of entity A.
   - Example: A `Professor` teaches many `Courses`, but a `Course` is taught by only one `Professor`.
3. **Many-to-Many (M:N)**: An instance of entity A can be related to many instances of entity B, and vice versa.[28]
   - Example: A `Student` can take many `Courses`, and a `Course` can have many `Students`.